

PRACTICAL 22 -

Objective - Objective - Design a stack that supports push, pop, top, and retrieving the minimum element in constant time. Implement push(), pop(), top(), and getMin() methods.

CODE -

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <limits.h>
4  #include <stdbool.h>
5
6  // Structure for the stack
7  struct Stack {
8      int top;
9      unsigned capacity;
10     int* array;
11     int* minStack; // Auxiliary stack to track minimums
12 };
13
14 // Function to create a stack
15 struct Stack* createStack(unsigned capacity) {
16     struct Stack* stack = (struct Stack*) malloc(sizeof(struct Stack));
17     stack->capacity = capacity;
18     stack->top = -1;
19     stack->array = (int*) malloc(stack->capacity * sizeof(int));
20     stack->minStack = (int*) malloc(stack->capacity * sizeof(int));
21     return stack;
22 }
23
24 // Function to check if the stack is empty
25 bool isEmpty(struct Stack* stack) {
26     return stack->top == -1;
27 }
28
29 // Function to check if the stack is full
30 bool isFull(struct Stack* stack) {
31     return stack->top == stack->capacity - 1;
32 }
33
34 // Function to push an element onto the stack
35 void push(struct Stack* stack, int data) {
36     if (isFull(stack))
37         return;
38     stack->array[++stack->top] = data;
39
40     // Update minStack
```

```
41     if (isEmpty(stack->minStack) || data <= stack->minStack[stack->
42         minStack->top]) {
43         stack->minStack[++stack->minStack->top] = data;
44     }
45 }
46 // Function to pop an element from the stack
47 int pop(struct Stack* stack) {
48     if (isEmpty(stack))
49         return INT_MIN; // Indicate underflow
50     int popped = stack->array[stack->top--];
51
52     // Update minStack
53     if (popped == stack->minStack[stack->minStack->top]) {
54         stack->minStack--;
55     }
56     return popped;
57 }
58
59 // Function to get the top element of the stack
60 int top(struct Stack* stack) {
61     if (isEmpty(stack))
62         return INT_MIN; // Indicate empty stack
63     return stack->array[stack->top];
64 }
65
66 // Function to get the minimum element in the stack
67 int getMin(struct Stack* stack) {
68     if (isEmpty(stack->minStack))
69         return INT_MIN; // Indicate empty stack
70     return stack->minStack[stack->minStack->top];
71 }
72
73 int main() {
74     struct Stack* stack = createStack(10);
75
76     push(stack, 5);
77     printf("Pushed: %d, Min: %d\n", 5, getMin(stack));
78     push(stack, 2);
79     printf("Pushed: %d, Min: %d\n", 2, getMin(stack));
80     push(stack, 8);
81     printf("Pushed: %d, Min: %d\n", 8, getMin(stack));
82     push(stack, 1);
83     printf("Pushed: %d, Min: %d\n", 1, getMin(stack));
84
85     printf("Top: %d\n", top(stack));
86
87     printf("Popped: %d, Min: %d\n", pop(stack), getMin(stack));
88     printf("Popped: %d, Min: %d\n", pop(stack), getMin(stack));
89     printf("Top: %d\n", top(stack));
90     printf("Min: %d\n", getMin(stack));
```

```
91  
92     return 0;  
93 }
```

OUTPUT -

```
Min: 3  
Min: 1  
Min: 2  
Top: 2
```